

Analytics Engineering Tech Challenge 2026

This challenge is designed to simulate a real, high-stakes analytics workflow at Auto1 Group.

We value correct logic, clear communication, and thorough data handling as much as code quality.

Please submit your answers as a ZIP file or private repository, including the code, documentation and output as specified.

Context

A stakeholder from the Customer Service department needs insights into cases where the **remarketing customer's IBAN (bank account details)** was changed after the company purchased* a vehicle from them.

Specifically, the stakeholder wants to know, **for each market and month in the past 12 months**, how often such post-purchase IBAN changes occurred.

The results should be emailed to the stakeholder as an excel attachment / shared via Google spreadsheet.

*Here, a “purchase” refers to an Auto1 Group entity buying a vehicle from a remarketing customer in a given country.

Data Access Constraints

Customers' bank account data is highly confidential (GDPR-sensitive), so it is:

- Encrypted
- Stored in a dedicated, restricted-access database
- Accessible only via a secured internal API.

To retrieve the bank account details for a given purchase, you must:

- Use the “bank_uuid” column from the “purchases” table,
- Pass it to the designated API endpoint.

For a small number of bank UUIDs, it is acceptable to retrieve the data through sequential API requests.

However, for thousands of UUIDs (as in this case), retrieving the data efficiently requires implementing asynchronous or parallelized bulk API calls - that is, sending multiple API requests in parallel rather than waiting for each to complete before starting the next.

Your mission

You need to architect your Python solution to efficiently and securely fetch bank details using asynchronous API requests to minimize total runtime, while considering API rate limits and error handling.

Your approach should handle partial failures (e.g., retries in case of failed batch requests), and guarantee that the report is complete and accurate. Document in your code / README how (and where) you log or report irrecoverable failures.

Provide your reasoning about pros and cons of available options and about the approach you selected. Point out any assumptions you make along the way.

Your solution needs to produce the required output and email it to / share it with the stakeholder.

Document your solution so that junior team members can understand it. Comments in code and markdown are highly valued.

If time allows, propose one extension or additional analysis you would suggest for the stakeholder based on these data sources.

Data Warehouse Tables

Below is a simplified set of tables, which reside in the Redshift schema “a1”, that are relevant for producing the output. **You need to** combine them in a SQL query to produce the output.

<purchases>

DDL:

```
CREATE TABLE purchases (  
    id BIGINT,  
    purchase_datetime TIMESTAMP WITHOUT TIME ZONE,  
    buy_price NUMERIC(11,2),  
    bank_uuid CHARACTER VARYING(32),  
    created_on TIMESTAMP WITHOUT TIME ZONE,  
    created_by BIGINT,  
    updated_on TIMESTAMP WITHOUT TIME ZONE,  
    ...  
);
```

Notes:

- bank_uuid can be null
- Number of records in purchases: tens of millions

<branches>

DDL:

```
CREATE TABLE branches (  
    id BIGINT,  
    name CHARACTER VARYING(765),  
    address CHARACTER VARYING(765),  
    country CHARACTER VARYING(6),  
    ...  
)  
SORTKEY(id);
```

Notes:

- Number of records in `branches`: tens of thousands
- `country` determines the market ('DE' for Germany, 'ES' for Spain, etc.)

<vehicles>

DDL:

```
CREATE TABLE vehicles (  
    id BIGINT,  
    code VARCHAR,  
    status_id BIGINT,  
    branch_id BIGINT,  
    dat_icode VARCHAR,  
    mileage BIGINT,  
    built_year INTEGER,  
    created_on TIMESTAMPTZ,  
    created_by BIGINT,  
    ...  
) ;
```

Notes:

- `id` corresponds to the `id` in the table `purchases`
- `branch_id` corresponds to the `id` in the table `branches`
- `branch_id` can be null
- Number of records in `vehicles`: hundreds of millions

<changelog_purchase_at_branch>

DDL:

```
CREATE TABLE changelog_purchase_at_branch (  
    id NUMERIC(20,0),  
    object_id BIGINT ENCODE AZ64,  
    field CHARACTER VARYING(135) ENCODE LZ0,  
    old_value CHARACTER VARYING(65535) ENCODE LZ0,
```

```

        new_value CHARACTER VARYING(65535) ENCODE LZO,
        created_at TIMESTAMP WITHOUT TIME ZONE,
        created_by BIGINT ENCODE AZ64
    )
    COMPOUND SORTKEY
    (
        id,
        created_at
    );

```

Notes:

- `id` is unique, it is a record identifier
- Value in the field `object_id` corresponds to the id of a purchase
- Field needs to be filtered for value “**financeUuid**” in order to see all the changes of bank details related to a purchase
- `old_value` keeps the value of the `financeUuid` field before the change happened
- `new_value` stores the new value of the `financeUuid` field, the one brought by the change
- `created_at` records the timestamp of the change of the `financeUuid` value
- ⚠ Any change to a bank account leads to a new value of the `financeUuid` i.e. an entry in the changelog table.
To discover if IBAN was changed, you need to make the API call to the restricted database, retrieve IBAN values and compare them.

Sample changelog_purchase_at_branch entries:

id	object	field	old_value	new_value	created_at	created_by
100001	5551111	financeUuid	f9c1c696-81d2-472a-93ba-cfd6803681a8	db62098a-8706-45cf-83ab-efc3ffeefb0d	2026-03-04 09:10:21	25510
135001	5551111	financeUuid	db62098a-8706-45cf-83ab-efc3ffeefb0d	4e35b2c2-dece-4bbf-92c6-e2cc89902efc	2026-04-25 15:31:43	55992
141275	7342101	financeUuid	NULL	0c31717b-07cb-4d68-8631-8ab1d96f7f0a	2026-02-13 13:00:57	1156992
274311	9140144	financeUuid	11aec7e8-7e42-4ebc-8ca7-cc6bdba0fcf6	2b41f143-36d8-4d11-8e86-bcbee0f88598	2026-05-14 10:01:14	
511230	9140144	financeUuid	2b41f143-36d8-4d11-8e86-bcbee0f88598	de402b89-3f15-4626-97c6-5c9e7987d066	2026-05-19 11:20:00	

Output Sample

You need to enlist only cases where the customer's IBAN was changed after the purchase. Please provide

- car_id
- branch_id
- country
- purchase_datetime
- the last 4 digits of IBAN value before the change event
- date and time when the above IBAN value was last active before the change
- change event date and time
- the last 4 digits of IBAN value after the change event

The overview needs to be done for each country (a worksheet per country).

origin	vehicles	branches		purchases		changelog_purchase_at_branch		from the api response	changelog_purchase_at_branch		from the api response
column	car_id	branch_id	country	purchase_datetime	current_bank_uuid	bank_uuid_right_before_purchase	bank_uuid_changed_before_purchase_at	iban_value_right_before_purchase	bank_uuid_after_purchase	bank_uuid_after_purchase_added_at	iban_value_after_purchase
sample data	12855084319	183	DE	2025-01-02 10:02:05	9a4321fb-20cd-489a-a4b3-840f8adf1ce6	f0e2d7c3-4b5a-4c96-9a0b-57dabb92d8e4	2025-01-02 09:02:05	1297	9a4321fb-20cd-489a-a4b3-840f8adf1ce6	2025-01-03 12:50:05	2058
	12896720531	524	DE	2025-01-02 10:42:58	64e8a15d-927b-4fb3-8305-cf98d64e10a3	ac827fa5-c4b3-4e8a-94c1-44be0f21837d	2025-01-02 09:42:58	5026	64e8a15d-927b-4fb3-8305-cf98d64e10a3	2025-01-10 10:51:23	4916
	12862479385	316	DE	2025-01-03 19:17:12	4b372a1c-67d1-4b86-8a0e-5ee2c8ef2b74	b3c2a41e-1e5a-4f36-8f95-5e24cf3db6b6	2025-01-03 18:17:12	7841	4b372a1c-67d1-4b86-8a0e-5ee2c8ef2b74	2025-01-08 07:30:06	7832
	12847593260	209	DE	2025-01-06 10:34:23	e5c900a7-4eae-495c-b7e1-b6b162832a10	f2bd3fac-e13a-4f4c-9282-9a04bd8a47fd	2025-01-06 09:34:23	3659	e5c900a7-4eae-495c-b7e1-b6b162832a10	2025-01-06 14:18:51	6179
	12880124697	67	DE	2025-01-06 14:10:09	7f1c5e90-08df-4e63-844d-3b7c6c7e01e1	6ec5b8f1-84ad-4c6a-b393-8c6c420f29e2	2025-01-06 13:10:09	2814	7f1c5e90-08df-4e63-844d-3b7c6c7e01e1	2025-01-07 11:54:01	3407
	12891536842	482	DE	2025-01-07 09:50:43	ba7e52ac-bd2b-492d-9567-3923dbfe8b1f	b7d041e0-59eb-41d3-9513-2e9d6d836168	2025-01-07 08:50:41	9405	ba7e52ac-bd2b-492d-9567-3923dbfe8b1f	2025-01-09 15:12:12	1592
	12830741956	135	DE	2025-01-07 11:28:10	5eb41cb3-11e7-435d-9440-3d57adc4af94	3fadbb12b-b27d-4e1f-bb19-9f3e1ec8c5c6	2025-01-07 10:28:09	4228	5eb41cb3-11e7-435d-9440-3d57adc4af94	2025-01-16 13:34:20	8240
	12874913086	298	DE	2025-01-07 11:38:35	c1c92abf-7e21-4dc6-b2fb-28c1fdf39ec7	ea6cd44f-af4a-4ae4-9370-f24b7cbb06e3	2025-01-07 10:38:34	8136	c1c92abf-7e21-4dc6-b2fb-28c1fdf39ec7	2025-01-15 11:22:06	0629
	12859387021	92	DE	2025-01-09 12:36:55	620b3e20-4872-470f-b6c2-4c834f67a9b9	7f48830b-8e50-44d1-98c9-c75561b0b5ad	2025-01-09 11:36:55	5372	620b3e20-4872-470f-b6c2-4c834f67a9b9	2025-01-13 09:42:22	4481

< >

Country 1

Country 2

Country 3

+

Technical API Information

Assumptions:

- **Headers**

```
api_header = {  
    "x-wa-auth": "123abcde123",  
    "x-wa-key": "123abcde123",  
    "authorization": "Bearer <token>", # You may assume the token is static for the  
    duration of the job.  
    "date": "Mon, 15 Dec 2200 09:43:0 UTC",  
}
```

- **Endpoint**

URL: "<http://xyz.internal.endpoint/search>"
HTTP Method: POST

- **API Request**

Send up to 1000 uuids per request in the payload.

- **API Response**

Note 1: Top-level field "results" contains a list of objects.

Note 2: Values are imaginary and have no relation to real data whatsoever.

Sample Response:

```
{  
  "results": [  
    {  
      "uuid": "a3907caf-51f0-4a95-9e58-756db1a323a2",  
      "created_on": "2026-05-20T08:00:00Z",  
      "account": {  
        "holder": "Alice Smith",  
        "iban": "DE89370400440532013000",  
        "bic": "COBADEFFXXX",  
        "bank_name": "Commerzbank"  
      }  
    },  
    {  
      "uuid": "1207c12f-c774-4618-a6f8-b7b3172e12f4",  
      "created_on": "2026-05-18T10:15:22Z",  
      "account": {  
        "holder": "Bob Johnson",
```

```
        "iban": "DE12500105170648489890",
        "bic": "INGDDEFFXXX",
        "bank_name": "ING"
    }
}
]
}
```

Response hierarchy mapping:

```
response_hierarchy = {
    'uuid': "",
    'created_on': "",
    'holder': 'account',
    'iban': 'account',
    'bic': 'account',
    'bank_name': 'account'
}
```

Fields can be located within a response following the above pattern.

Value "" indicates that the field is at the root level, otherwise it gives the object path.